



Faculty of Computer Applications

BCA Sem-1 (A) (AY-2026-27)

**Sub Code: 05BC3103
(Programming Practices: 1)**

List of experiments with Advanced/Design/Open ended experiments

Roll No :	92600527073
Full Name :	Siddhi .S. Gangani

GitHub Link | <https://github.com/siddhigangani9>

**Sub. Faculty Name:
Prof. Vipulkumar Bambhaniya**

Write a C Program with proper comment and indentation with Output

1) Student Performance Analysis System

Problem Definition:

Develop a menu-driven C application to analyze the academic performance of a group of students. The system should accept student information such as roll number, name, marks obtained in different subjects, and attendance percentage. Based on the available data, the application should calculate total marks, percentage, grade, pass/fail status, class average, highest and lowest performers, and other meaningful performance indicators.

The program should allow the user to add, display, search, and analyze student records. Students should decide the appropriate use of arrays, structures, functions, conditional statements, loops, and other C programming concepts.

Minimum Requirements:

- Accept details of multiple students.
- Calculate total and percentage.
- Assign grades based on suitable criteria.
- Identify pass/fail students.
- Find highest and lowest performers.
- Calculate average performance.
- Provide a menu-driven interface.

Open-Ended Aspect:

Students can independently decide the grading criteria, data representation, functions, searching method, sorting technique, and additional performance indicators.

Input:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_STUDENTS 100
```

```
#define NUM_SUBJECTS 3
```

```
// Structure to store student information
```

```
typedef struct {  
    int rollNumber;  
    char name[50];  
    float marks[NUM_SUBJECTS];  
    float attendance;  
    float totalMarks;  
    float percentage;  
    char grade;  
    char status[5]; // "Pass" or "Fail"  
} Student;  
  
// Function prototypes  
void addStudents(Student students[], int *count);  
void displayStudents(const Student students[], int count);  
void searchStudent(const Student students[], int count);  
void analyzePerformance(const Student students[], int count);  
void sortStudentsByPercentage(Student students[], int count);  
void calculateMetrics(Student *s);  
char assignGrade(float percentage);  
  
int main() {  
    Student students[MAX_STUDENTS];  
    int studentCount = 0;
```

```

int choice;

// Menu-driven loop
do {
    printf("\n=====\\n");
    printf("  STUDENT PERFORMANCE ANALYSIS SYSTEM \\n");
    printf("=====\\n");
    printf("1. Add Student Records\\n");
    printf("2. Display All Student Records\\n");
    printf("3. Search Student by Roll Number\\n");
    printf("4. Analyze & View Performance Metrics\\n");
    printf("5. Sort Records by Merit (Percentage)\\n");
    printf("6. Exit\\n");
    printf("-----\\n");
    printf("Enter your choice (1-6): ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            addStudents(students, &studentCount);
            break;
        case 2:
            displayStudents(students, studentCount);

```

```

        break;

    case 3:

        searchStudent(students, studentCount);

        break;

    case 4:

        analyzePerformance(students, studentCount);

        break;

    case 5:

        sortStudentsByPercentage(students, studentCount);

        printf("\n[Success] Records successfully sorted by merit!\n");

        break;

    case 6:

        printf("\nExiting program. Thank you!\n");

        break;

    default:

        printf("\n[Error] Invalid choice! Please select between 1 and 6.\n");

    }

} while (choice != 6);

return 0;

}

// Function to calculate independent student statistics

```

```

void calculateMetrics(Student *s) {

    s->totalMarks = 0;

    int failedSubject = 0;

    for (int i = 0; i < NUM_SUBJECTS; i++) {

        s->totalMarks += s->marks[i];

        if (s->marks[i] < 40.0) { // Sub-criterion: failing any subject drops status to Fail

            failedSubject = 1;

        }

    }

    s->percentage = s->totalMarks / NUM_SUBJECTS;

    s->grade = assignGrade(s->percentage);

    // Pass criteria: Average >= 40% AND no individual subject marks < 40
    if (s->percentage >= 40.0 && !failedSubject) {

        strcpy(s->status, "Pass");

    } else {

        strcpy(s->status, "Fail");

    }

}

// Function to dynamically assign grades based on percentage thresholds

```

```
char assignGrade(float percentage) {  
    if (percentage >= 85.0) return 'A';  
    if (percentage >= 70.0) return 'B';  
    if (percentage >= 55.0) return 'C';  
    if (percentage >= 40.0) return 'D';  
    return 'F';  
}
```

// Function to add new student profiles to the memory array

```
void addStudents(Student students[], int *count) {  
    int num;  
    printf("\nHow many students do you want to add? ");  
    scanf("%d", &num);  
  
    for (int i = 0; i < num; i++) {  
        if (*count >= MAX_STUDENTS) {  
            printf("\n[System Alert] Maximum database limit reached!\n");  
            return;  
        }  
  
        printf("\n--- Entering Details for Student %d ---\n", *count + 1);  
        printf("Enter Roll Number: ");  
        scanf("%d", &students[*count].rollNumber);  
    }  
}
```

```

    getchar(); // Clear trailing newline buffer character

    printf("Enter Name: ");

    fgets(students[*count].name, sizeof(students[*count].name), stdin);

    students[*count].name[strcspn(students[*count].name, "\n")] = 0; // Strip newline

    printf("Enter Marks for %d Subjects (out of 100):\n", NUM_SUBJECTS);

    for (int j = 0; j < NUM_SUBJECTS; j++) {

        printf(" Subject %d: ", j + 1);

        scanf("%f", &students[*count].marks[j]);

    }

    printf("Enter Attendance Percentage: ");

    scanf("%f", &students[*count].attendance);

    // Auto-compute secondary metrics

    calculateMetrics(&students[*count]);

    (*count)++;

}

printf("\n[Success] Records added securely.\n");

}

// Function to print all students in a neat tabular matrix

```

```

void displayStudents(const Student students[], int count) {
    if (count == 0) {
        printf("\n[System Info] No records available to display.\n");
        return;
    }

    printf("\n=====
    =====\n");

    printf("%-7s %-20s %-8s %-12s %-7s %-6s %-6s\n", "Roll No", "Name", "Total",
    "Percentage", "Attend%%", "Grade", "Status");

    printf("=====
    =====\n");

    for (int i = 0; i < count; i++) {
        printf("%-7d %-20s %-8.1f %-11.2f%% %-7.1f%% %-6c %-6s\n",
            students[i].rollNumber, students[i].name, students[i].totalMarks,
            students[i].percentage, students[i].attendance, students[i].grade,
            students[i].status);
    }

    printf("=====
    =====\n");
}

// Function to search for an individual record using linear search
void searchStudent(const Student students[], int count) {

```

```

if (count == 0) {

    printf("\n[System Info] Database empty. Cannot perform search.\n");

    return;

}

int roll, found = 0;

printf("\nEnter Roll Number to search: ");

scanf("%d", &roll);

for (int i = 0; i < count; i++) {

    if (students[i].rollNumber == roll) {

        found = 1;

        printf("\n--- Record Found! ---\n");

        printf("Roll Number: %d\n", students[i].rollNumber);

        printf("Name:    %s\n", students[i].name);

        printf("Attendance: %.2f%%\n", students[i].attendance);

        printf("Final Grade: %c (%s)\n", students[i].grade, students[i].status);

        printf("Subject Breakdowns:\n");

        for (int j = 0; j < NUM_SUBJECTS; j++) {

            printf("  Subject %d Score: %.1f\n", j + 1, students[i].marks[j]);

        }

        break;

    }

}

```

```

    }

    if (!found) {
        printf("\n[Error] Student with Roll Number %d not found.\n", roll);
    }
}

// Function to evaluate analytical performance indices
void analyzePerformance(const Student students[], int count) {
    if (count == 0) {
        printf("\n[System Info] No data available for global statistical metrics.\n");
        return;
    }

    float totalClassSum = 0, highestPerc = -1.0, lowestPerc = 101.0;
    int highestIdx = 0, lowestIdx = 0;
    int passCount = 0;

    for (int i = 0; i < count; i++) {
        totalClassSum += students[i].percentage;

        if (students[i].percentage > highestPerc) {
            highestPerc = students[i].percentage;
            highestIdx = i;

```

```

    }

    if (students[i].percentage < lowestPerc) {

        lowestPerc = students[i].percentage;

        lowestIdx = i;
    }

    if (strcmp(students[i].status, "Pass") == 0) {

        passCount++;
    }
}

printf("\n=====\\n");

printf("    ACADEMIC METRICS SUMMARY    \\n");

printf("=====\\n");

printf("Class Size      : %d\\n", count);

printf("Class Batch Average : %.2f%%\\n", totalClassSum / count);

printf("Overall Pass Rate  : %.1f%%\\n", ((float)passCount / count) * 100);

printf("-----\\n");

printf("Highest Performer   : %s (Roll: %d) with %.2f%%\\n",

    students[highestIdx].name, students[highestIdx].rollNumber,
    students[highestIdx].percentage);

printf("Lowest Performer    : %s (Roll: %d) with %.2f%%\\n",

    students[lowestIdx].name, students[lowestIdx].rollNumber,
    students[lowestIdx].percentage);

printf("=====\\n");

```

```
}
```

```
// Function sorting records descendingly via standard Selection Sort logic
```

```
void sortStudentsByPercentage(Student students[], int count) {  
    for (int i = 0; i < count - 1; i++) {  
        int maxIdx = i;  
        for (int j = i + 1; j < count; j++) {  
            if (students[j].percentage > students[maxIdx].percentage) {  
                maxIdx = j;  
            }  
        }  
        if (maxIdx != i) {  
            Student temp = students[i];  
            students[i] = students[maxIdx];  
            students[maxIdx] = temp;  
        }  
    }  
}
```

Output:

```

1. Add Student Records
2. Display All Student Records
3. Search Student by Roll Number
4. Analyze & View Performance Metrics
5. Sort Records by Merit (Percentage)
6. Exit
-----
Enter your choice (1-6): 1

How many students do you want to add? 1

--- Entering Details for Student 1 ---
Enter Roll Number: 1
Enter Name: siddhi
Enter Marks for 3 Subjects (out of 100):
  Subject 1: 78
  Subject 2: 67
  Subject 3: 79
Enter Attendance Percentage: 95

[Success] Records added securely.

=====
      STUDENT PERFORMANCE ANALYSIS SYSTEM
=====
1. Add Student Records
2. Display All Student Records
3. Search Student by Roll Number
4. Analyze & View Performance Metrics
5. Sort Records by Merit (Percentage)
6. Exit
-----
Enter your choice (1-6): 2

=====
Roll No Name                Total    Percentage  Attend% Grade  Status
=====
1      siddhi                224.0    74.67      % 95.0  % B    Pass
=====

```

2) Library Management System

Problem Definition:

Design and implement a C-based library management system that maintains information about books available in a library. The system should provide facilities for adding book records, displaying available books, searching for a particular book, issuing and returning books, and identifying currently available or issued books.

Each book may contain information such as book ID, title, author, category, price, and availability status. The student should determine an appropriate representation of book information and develop suitable functions for different operations.

Minimum Requirements:

- Add book information.
- Display all books.
- Search books by ID, title, or author.
- Issue a book.
- Return a book.
- Display available books.
- Display issued books.

Open-Ended Aspect:

Students may extend the system with fine calculation, sorting, file storage, multiple search options, and issue-date information.

Input:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#define MAX_BOOKS 100
```

```
struct Book {
```

```
    int id;
```

```
    char title[50];
```

```
    char author[50];
```

```
    char category[30];
```

```
    float price;
```

```
    int isIssued; // 0 for available, 1 for issued
```

```
};
```

```
struct Book library[MAX_BOOKS];
```

```
int bookCount = 0;
```

```
// Function prototypes
```

```
void addBook();
```

```
void displayAllBooks();

void searchBook();

void issueBook();

void returnBook();

void displayAvailableBooks();

void displayIssuedBooks();

void loadFromFile();

void saveToFile();


void addBook() {
    if (bookCount >= MAX_BOOKS) {
        printf("\nLibrary database is full!\n");
        return;
    }

    struct Book b;

    b.id = bookCount + 1; // Auto-increment ID or manual input
    printf("\nEnter Book Title: ");
    getchar(); // clear newline
    fgets(b.title, sizeof(b.title), stdin);
    b.title[strcspn(b.title, "\n")] = 0;


    printf("Enter Author Name: ");
    fgets(b.author, sizeof(b.author), stdin);
```

```

b.author[strcspn(b.author, "\n")] = 0;

printf("Enter Category: ");
fgets(b.category, sizeof(b.category), stdin);
b.category[strcspn(b.category, "\n")] = 0;

printf("Enter Price: ");
scanf("%f", &b.price);

b.isIssued = 0;
library[bookCount++] = b;
saveToFile();
printf("\nBook added successfully! Assigned ID: %d\n", b.id);
}

void displayAllBooks() {
    if (bookCount == 0) {
        printf("\nNo books available in the library.\n");
        return;
    }

    printf("\n--- All Book Records ---\n");
    for (int i = 0; i < bookCount; i++) {
        printf("ID: %d | Title: %s | Author: %s | Category: %s | Price: %.2f | Status: %s\n",

```

```
        library[i].id, library[i].title, library[i].author,  
        library[i].category, library[i].price,  
        library[i].isIssued ? "Issued" : "Available");  
    }  
}
```

```
void searchBook() {  
    if (bookCount == 0) {  
        printf("\nNo books to search.\n");  
        return;  
    }  
  
    int choice, id, found = 0;  
    char query[50];  
  
    printf("\nSearch by:\n1. ID\n2. Title\n3. Author\nEnter choice: ");  
    scanf("%d", &choice);  
  
    if (choice == 1) {  
        printf("Enter Book ID: ");  
        scanf("%d", &id);  
        for (int i = 0; i < bookCount; i++) {  
            if (library[i].id == id) {  
                printf("Found: ID: %d | Title: %s | Author: %s | Status: %s\n",
```

```

        library[i].id, library[i].title, library[i].author, library[i].isIssued ? "Issued" :
"Available");

        found = 1;

        break;

    }

}

} else {

    printf("Enter search query: ");

    getchar();

    fgets(query, sizeof(query), stdin);

    query[strcspn(query, "\n")] = 0;


    for (int i = 0; i < bookCount; i++) {

        if ((choice == 2 && strcasecmp(library[i].title, query) == 0) ||

            (choice == 3 && strcasecmp(library[i].author, query) == 0)) {

            printf("Found: ID: %d | Title: %s | Author: %s | Status: %s\n",

                library[i].id, library[i].title, library[i].author, library[i].isIssued ? "Issued" :
"Available");

            found = 1;

        }

    }

}

if (!found) printf("\nNo matching book found.\n");

}

```

```
void issueBook() {  
    int id;  
    printf("Enter Book ID to issue: ");  
    scanf("%d", &id);  
    for (int i = 0; i < bookCount; i++) {  
        if (library[i].id == id) {  
            if (library[i].isIssued) {  
                printf("\nBook is already issued!\n");  
            } else {  
                library[i].isIssued = 1;  
                saveToFile();  
                printf("\nBook issued successfully!\n");  
            }  
            return;  
        }  
    }  
    printf("\nBook ID not found.\n");  
}
```

```
void returnBook() {  
    int id;  
    printf("Enter Book ID to return: ");
```

```

scanf("%d", &id);

for (int i = 0; i < bookCount; i++) {

    if (library[i].id == id) {

        if (!library[i].isIssued) {

            printf("\nBook was not issued!\n");

        } else {

            library[i].isIssued = 0;

            saveToFile();

            printf("\nBook returned successfully!\n");

        }

        return;

    }

}

printf("\nBook ID not found.\n");

}

void displayAvailableBooks() {

    printf("\n--- Available Books ---\n");

    int count = 0;

    for (int i = 0; i < bookCount; i++) {

        if (!library[i].isIssued) {

            printf("ID: %d | Title: %s | Author: %s\n", library[i].id, library[i].title,
library[i].author);

            count++;


```

```

    }
}

if (count == 0) printf("No available books.\n");
}

void displayIssuedBooks() {
    printf("\n--- Issued Books ---\n");
    int count = 0;
    for (int i = 0; i < bookCount; i++) {
        if (library[i].isIssued) {
            printf("ID: %d | Title: %s | Author: %s\n", library[i].id, library[i].title,
library[i].author);
            count++;
        }
    }
    if (count == 0) printf("No issued books.\n");
}

void saveToFile() {
    FILE *file = fopen("library.dat", "wb");
    if (file) {
        fwrite(&bookCount, sizeof(int), 1, file);
        fwrite(library, sizeof(struct Book), bookCount, file);
        fclose(file);
    }
}

```

```

    }
}

void loadFromFile() {
    FILE *file = fopen("library.dat", "rb");

    if (file) {
        fread(&bookCount, sizeof(int), 1, file);

        fread(library, sizeof(struct Book), bookCount, file);

        fclose(file);
    }
}

```

```

int main() {
    loadFromFile();

    int choice;

    do {
        printf("\n=== Library Management System ===\n");

        printf("1. Add Book\n2. Display All Books\n3. Search Book\n4. Issue Book\n5. Return Book\n6. Display Available Books\n7. Display Issued Books\n8. Exit\n");

        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch (choice) {
            case 1: addBook(); break;

            case 2: displayAllBooks(); break;

```

```

        case 3: searchBook(); break;

        case 4: issueBook(); break;

        case 5: returnBook(); break;

        case 6: displayAvailableBooks(); break;

        case 7: displayIssuedBooks(); break;

        case 8: printf("\nExiting program.\n"); break;

        default: printf("\nInvalid choice! Try again.\n");

    }

} while (choice != 8);

return 0;

}

```

Output:

```

=== Library Management System ===
. Add Book
. Display All Books
. Search Book
. Issue Book
. Return Book
. Display Available Books
. Display Issued Books
. Exit
Enter your choice: 1

Enter Book Title: don't believe everything you think
Enter Author Name: joseph nguyen
Enter Category: personal development
Enter Price: 240

Book added successfully! Assigned ID: 1

=== Library Management System ===
. Add Book
. Display All Books
. Search Book
. Issue Book
. Return Book
. Display Available Books
. Display Issued Books
. Exit
Enter your choice: 2

-- All Book Records --
ID: 1 | Title: don't believe everything you think | Author: joseph nguyen | Category: personal development | Price: 240.00 | Status: Available

=== Library Management System ===
. Add Book
. Display All Books

```

3) Inventory Management and Billing System

Problem Definition:

Develop a C program to manage the inventory of a small shop. The application should maintain product details such as product ID, name, category, quantity, unit price, and stock status. The system should allow the user to add products, update stock, search products, generate customer bills, and identify products whose stock has reached a predefined minimum level.

The billing component should allow the user to purchase multiple products and calculate the total bill based on quantity and price. Students should determine how the inventory and billing information will be represented and processed.

Minimum Requirements:

- Add and display products.
- Search for products.
- Update stock quantity.
- Generate a bill.
- Calculate total amount.
- Identify low-stock products.

Open-Ended Aspect:

Students may implement discounts, taxes, customer information, bill numbers, file handling, sorting, and sales reports.

Input:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#define MAX_PRODUCTS 100
```

```
#define MIN_STOCK_LEVEL 5
```

```
#define FILE_NAME "inventory.dat"
```

```
struct Product {
```

```

    int id;

    char name[50];

    char category[30];

    int quantity;

    float unitPrice;

    char stockStatus[20];
};

void updateStockStatus(struct Product *p) {
    if (p->quantity <= 0) {
        strcpy(p->stockStatus, "Out of Stock");
    } else if (p->quantity <= MIN_STOCK_LEVEL) {
        strcpy(p->stockStatus, "Low Stock");
    } else {
        strcpy(p->stockStatus, "In Stock");
    }
}

void saveToFile(struct Product products[], int count) {
    FILE *file = fopen(FILE_NAME, "wb");

    if (file != NULL) {
        fwrite(&count, sizeof(int), 1, file);

        fwrite(products, sizeof(struct Product), count, file);
    }
}

```

```
        fclose(file);
    }
}

int loadFromFile(struct Product products[]) {
    int count = 0;
    FILE *file = fopen(FILE_NAME, "rb");
    if (file != NULL) {
        fread(&count, sizeof(int), 1, file);
        fread(products, sizeof(struct Product), count, file);
        fclose(file);
    }
    return count;
}
```

```
void addProduct(struct Product products[], int *count) {
    if (*count >= MAX_PRODUCTS) {
        printf("Inventory is full!\n");
        return;
    }
    struct Product p;
    p.id = *count + 1; // Auto-generate ID
    printf("Enter Product Name: ");
```

```

scanf(" %[^\\n]s", p.name);

printf("Enter Category: ");

scanf(" %[^\\n]s", p.category);

printf("Enter Quantity: ");

scanf("%d", &p.quantity);

printf("Enter Unit Price: ");

scanf("%f", &p.unitPrice);


updateStockStatus(&p);

products[*count] = p;

(*count)++;

saveToFile(products, *count);

printf("Product added successfully! Assigned ID: %d\\n", p.id);
}


void displayProducts(struct Product products[], int count) {

    if (count == 0) {

        printf("No products in inventory.\\n");

        return;

    }

    printf("\\n%-5s %-20s %-15s %-10s %-12s %-15s\\n", "ID", "Name", "Category", "Qty",
"Price", "Status");

    printf("-----\\n");

    for (int i = 0; i < count; i++) {

```

```

        updateStockStatus(&products[i]);

        printf("%-5d %-20s %-15s %-10d %-12.2f %-15s\n",
               products[i].id, products[i].name, products[i].category,
               products[i].quantity, products[i].unitPrice, products[i].stockStatus);
    }
}

void searchProduct(struct Product products[], int count) {
    int choice, id, found = 0;
    char name[50];
    printf("Search by: 1. ID 2. Name\nEnter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("Enter Product ID: ");
        scanf("%d", &id);
        for (int i = 0; i < count; i++) {
            if (products[i].id == id) {
                printf("\nFound: ID: %d | Name: %s | Qty: %d | Price: %.2f | Status: %s\n",
                       products[i].id, products[i].name, products[i].quantity, products[i].unitPrice,
                       products[i].stockStatus);

                found = 1;
                break;
            }
        }
    }
}

```

```

    }
} else {
    printf("Enter Product Name: ");
    scanf(" %[^\\n]s", name);
    for (int i = 0; i < count; i++) {
        if (strcasecmp(products[i].name, name) == 0) {
            printf("\\nFound: ID: %d | Name: %s | Qty: %d | Price: %.2f | Status: %s\\n",
                products[i].id, products[i].name, products[i].quantity, products[i].unitPrice,
                products[i].stockStatus);
            found = 1;
        }
    }
}

if (!found) printf("Product not found.\\n");
}

```

```

void updateStock(struct Product products[], int count) {
    int id, qtyChange;
    printf("Enter Product ID to update stock: ");
    scanf("%d", &id);
    for (int i = 0; i < count; i++) {
        if (products[i].id == id) {
            printf("Current Quantity: %d\\nEnter quantity to add (positive) or remove
(negative): ", products[i].quantity);

```

```

        scanf("%d", &qtyChange);

        products[i].quantity += qtyChange;

        if (products[i].quantity < 0) products[i].quantity = 0;

        updateStockStatus(&products[i]);

        saveToFile(products, count);

        printf("Stock updated successfully! New Qty: %d\n", products[i].quantity);

        return;
    }
}

printf("Product ID not found.\n");
}

void checkLowStock(struct Product products[], int count) {
    int found = 0;

    printf("\n--- Low Stock / Out of Stock Alerts ---\n");

    for (int i = 0; i < count; i++) {
        updateStockStatus(&products[i]);

        if (products[i].quantity <= MIN_STOCK_LEVEL) {
            printf("ID: %d | Name: %s | Qty Left: %d | Status: %s\n",
                products[i].id, products[i].name, products[i].quantity, products[i].stockStatus);

            found = 1;
        }
    }
}

```

```

    if (!found) printf("All stock levels are adequate.\n");
}

void generateBill(struct Product products[], int count) {
    int id, qty, more = 1;
    float grandTotal = 0;

    printf("\n===== CUSTOMER BILL =====\n");
    printf("%-20s %-10s %-12s %-12s\n", "Item", "Qty", "Price", "Total");
    printf("-----\n");

    while (more) {
        printf("Enter Product ID to purchase: ");
        scanf("%d", &id);
        printf("Enter Quantity: ");
        scanf("%d", &qty);

        int found = 0;
        for (int i = 0; i < count; i++) {
            if (products[i].id == id) {
                found = 1;
                if (products[i].quantity >= qty) {
                    products[i].quantity -= qty;

```

```

        float totalCost = qty * products[i].unitPrice;

        grandTotal += totalCost;

        printf("%-20s %-10d %-12.2f %-12.2f\n", products[i].name, qty,
products[i].unitPrice, totalCost);

        updateStockStatus(&products[i]);

    } else {

        printf("Insufficient stock! Available: %d\n", products[i].quantity);

    }

    break;

}

}

if (!found) printf("Invalid Product ID.\n");


printf("Add another item? (1 for Yes, 0 for No): ");

scanf("%d", &more);

}


saveToFile(products, count);

printf("-----\n");

printf("GRAND TOTAL: %.2f\n", grandTotal);

printf("=====\n");

}

int main() {

```

```
struct Product products[MAX_PRODUCTS];

int count = loadFromFile(products);

int choice;

do {

    printf("\n--- Shop Inventory & Billing System ---\n");

    printf("1. Add Product\n");

    printf("2. Display Products\n");

    printf("3. Search Product\n");

    printf("4. Update Stock\n");

    printf("5. Check Low Stock\n");

    printf("6. Generate Bill\n");

    printf("7. Exit\n");

    printf("Enter your choice: ");

    scanf("%d", &choice);

    switch (choice) {

        case 1: addProduct(products, &count); break;

        case 2: displayProducts(products, count); break;

        case 3: searchProduct(products, count); break;

        case 4: updateStock(products, count); break;

        case 5: checkLowStock(products, count); break;

        case 6: generateBill(products, count); break;
```

```

        case 7: printf("Exiting program.\n"); break;

        default: printf("Invalid choice! Try again.\n");

    }

} while (choice != 7);

return 0;

}

```

Output:

```

--- Shop Inventory & Billing System ---
1. Add Product
2. Display Products
3. Search Product
4. Update Stock
5. Check Low Stock
6. Generate Bill
7. Exit
Enter your choice: 1
Enter Product Name: fuse
Enter Category: chocolate
Enter Quantity: 5
Enter Unit Price: 10
Product added successfully! Assigned ID: 1

--- Shop Inventory & Billing System ---
1. Add Product
2. Display Products
3. Search Product
4. Update Stock
5. Check Low Stock
6. Generate Bill
7. Exit
Enter your choice: 6

===== CUSTOMER BILL =====
Item          Qty      Price      Total
-----
Enter Product ID to purchase: 1
Enter Quantity: 5
fuse          5        10.00      50.00
Add another item? (1 for Yes, 0 for No): 0
-----
GRAND TOTAL: 50.00
=====

--- Shop Inventory & Billing System ---

```

4) Student Attendance Management System

Problem Definition:

Develop a C application for maintaining and analyzing student attendance. The system should accept attendance information for multiple students over a specified period and calculate individual attendance percentages.

The program should identify students who satisfy or fail to satisfy the required attendance criteria. It should also generate summary information such as class attendance percentage, students with maximum attendance, students with minimum attendance, and students requiring improvement.

Minimum Requirements:

- Store student attendance information.
- Calculate attendance percentage.
- Determine eligibility based on a specified threshold.
- Display students below the threshold.
- Generate attendance summary.

Open-Ended Aspect:

Students can decide whether to represent attendance using arrays, structures, or files and can introduce monthly/subject-wise attendance analysis.

Input:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_STUDENTS 50
```

```
#define NAME_LEN 50
```

```
struct Student {  
    int id;  
    char name[NAME_LEN];  
    int classes_attended;  
    int total_classes;  
    float percentage;  
};
```

```
// Function to calculate attendance percentage
```

```

void calculatePercentage(struct Student *s) {
    if (s->total_classes > 0) {
        s->percentage = ((float)s->classes_attended / s->total_classes) * 100.0;
    } else {
        s->percentage = 0.0;
    }
}

int main() {
    struct Student students[MAX_STUDENTS];
    int n, i;
    float threshold, class_total_attended = 0, class_total_classes = 0, class_percentage = 0;

    printf("=== Student Attendance Management System ===\n");
    printf("Enter total number of students (max %d): ", MAX_STUDENTS);
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAX_STUDENTS) {
        printf("Invalid number of students.\n");
        return 1;
    }

    printf("Enter minimum attendance threshold percentage (e.g., 75.0): ");
    scanf("%f", &threshold);

    // Input student details
    for (i = 0; i < n; i++) {
        printf("\nEnter details for Student %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &students[i].id);
        printf("Name: ");
        scanf("%s", students[i].name);
    }
}

```

```

printf("Classes Attended: ");
scanf("%d", &students[i].classes_attended);
printf("Total Classes Conducted: ");
scanf("%d", &students[i].total_classes);

calculatePercentage(&students[i]);

class_total_attended += students[i].classes_attended;
class_total_classes += students[i].total_classes;
}

// Calculate overall class percentage
if (class_total_classes > 0) {
    class_percentage = (class_total_attended / class_total_classes) * 100.0;
}

// Find max, min attendance, and students requiring improvement
int max_idx = 0, min_idx = 0;
for (i = 1; i < n; i++) {
    if (students[i].percentage > students[max_idx].percentage) {
        max_idx = i;
    }
    if (students[i].percentage < students[min_idx].percentage) {
        min_idx = i;
    }
}

// Display Summary Report
printf("\n\n=== ATTENDANCE SUMMARY REPORT ===\n");
printf("Overall Class Attendance Percentage: %.2f%%\n", class_percentage);

```

```

printf("\nStudent with Maximum Attendance:\n");
printf("ID: %d, Name: %s (%.2f%%)\n", students[max_idx].id,
students[max_idx].name, students[max_idx].percentage);

printf("\nStudent with Minimum Attendance:\n");
printf("ID: %d, Name: %s (%.2f%%)\n", students[min_idx].id,
students[min_idx].name, students[min_idx].percentage);

printf("\nStudents Below Threshold (Eligible for improvement/Defaulters):\n");
int below_count = 0;
for (i = 0; i < n; i++) {
    if (students[i].percentage < threshold) {
        printf("- ID: %d, Name: %s | Attendance: %.2f%% (Status: Deficit)\n",
            students[i].id, students[i].name, students[i].percentage);
        below_count++;
    }
}
if (below_count == 0) {
    printf("None! All students satisfy the attendance criteria.\n");
}

return 0;
}

```

Output:

```
=== Student Attendance Management System ===
Enter total number of students (max 50): 2
Enter minimum attendance threshold percentage (e.g., 75.0): 75

Enter details for Student 1:
ID: 1
Name: siddhi
Classes Attended: 40
Total Classes Conducted: 45

Enter details for Student 2:
ID: 2
Name: angel
Classes Attended: 39
Total Classes Conducted: 45

=== ATTENDANCE SUMMARY REPORT ===
Overall Class Attendance Percentage: 87.78%

Student with Maximum Attendance:
ID: 1, Name: siddhi (88.89%)

Student with Minimum Attendance:
ID: 2, Name: angel (86.67%)

Students Below Threshold (Eligible for improvement/Defaulters):
None! All students satisfy the attendance criteria.

...Program finished with exit code 0
Press ENTER to exit console.
```

5) Employee Payroll Management System

Problem Definition:

Develop a C-based payroll application for an organization. The system should maintain employee information and calculate the gross salary, deductions, and net salary of each employee.

The application should accept employee details such as employee ID, name, designation, basic salary, allowances, and deductions. Based on suitable salary rules, the program should generate individual salary details and organizational salary summaries.

Minimum Requirements:

- Store employee information.
- Calculate allowances.
- Calculate gross salary.
- Calculate deductions.
- Calculate net salary.
- Display employee salary details.
- Identify employees based on salary criteria.

Open-Ended Aspect:

Students may define suitable salary slabs, introduce different employee categories, generate salary reports, and implement file handling.

Input:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
// Define a structure to store employee payroll details
```

```
struct Employee {
```

```
    int id;
```

```
    char name[50];
```

```
    char designation[50];
```

```
    float basicSalary;
```

```
    float hra;    // House Rent Allowance (15% of Basic)
```

```
    float da;    // Dearness Allowance (20% of Basic)
```

```
    float pf;    // Provident Fund (10% of Basic)
```

```
    float lic;    // Life Insurance Corporation (8% of Basic)
```

```
    float grossSalary;
```

```
    float totalDeductions;
```

```
    float netSalary;
```

```
};
```

```
// Function to calculate salary components
```

```
void calculateSalary(struct Employee *emp) {
```

```
    // Calculate Allowances
```

```

emp->hra = 0.15 * emp->basicSalary;

emp->da = 0.20 * emp->basicSalary;


// Calculate Gross Salary = Basic + HRA + DA
emp->grossSalary = emp->basicSalary + emp->hra + emp->da;


// Calculate Deductions = PF (10%) + LIC (8%)
emp->pf = 0.10 * emp->basicSalary;
emp->lic = 0.08 * emp->basicSalary;
emp->totalDeductions = emp->pf + emp->lic;


// Calculate Net Salary = Gross Salary - Total Deductions
emp->netSalary = emp->grossSalary - emp->totalDeductions;
}


// Function to display individual salary slip
void displayPaySlip(struct Employee emp) {
    printf("\n-----\n");
    printf("      EMPLOYEE PAY SLIP      \n");
    printf("-----\n");
    printf("Employee ID   : %d\n", emp.id);
    printf("Name         : %s\n", emp.name);
    printf("Designation   : %s\n", emp.designation);

```

```

printf("Basic Salary : %.2f\n", emp.basicSalary);
printf("HRA (15%%)   : %.2f\n", emp.hra);
printf("DA (20%%)   : %.2f\n", emp.da);
printf("Gross Salary : %.2f\n", emp.grossSalary);
printf("PF (10%%)   : %.2f\n", emp.pf);
printf("LIC (8%%)   : %.2f\n", emp.lic);
printf("Total Ded.  : %.2f\n", emp.totalDeductions);
printf("Net Salary  : %.2f\n", emp.netSalary);
printf("-----\n");
}

```

```

int main() {
    int n, i;

    printf("Enter the number of employees: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid input.\n");
        return 1;
    }
}

```

```

struct Employee emp[n];

float orgTotalGross = 0, orgTotalNet = 0;

```

```
// Input Employee Details

for (i = 0; i < n; i++) {

    printf("\nEnter details for Employee %d:\n", i + 1);

    printf("Employee ID: ");

    scanf("%d", &emp[i].id);


    printf("Name: ");

    scanf(" %[^\\n]s", emp[i].name); // Read string with spaces


    printf("Designation: ");

    scanf(" %[^\\n]s", emp[i].designation);


    printf("Basic Salary: ");

    scanf("%f", &emp[i].basicSalary);


    // Perform calculations

    calculateSalary(&emp[i]);


    // Accumulate organizational totals

    orgTotalGross += emp[i].grossSalary;

    orgTotalNet += emp[i].netSalary;

}
```

```

// Generate Individual Salary Slips

for (i = 0; i < n; i++) {
    displayPaySlip(emp[i]);
}

// Generate Organizational Salary Summary

printf("\n=====\\n");
printf("    ORGANIZATIONAL SALARY SUMMARY    \\n");
printf("=====\\n");
printf("Total Employees Processed : %d\\n", n);
printf("Total Gross Payout      : %.2f\\n", orgTotalGross);
printf("Total Net Payout        : %.2f\\n", orgTotalNet);
printf("=====\\n");

return 0;
}

```

Output:

```
Enter the number of employees: 1

Enter details for Employee 1:
Employee ID: 3
Name: sid
Designation: manager
Basic Salary: 100000

-----
EMPLOYEE PAY SLIP
-----
Employee ID   : 3
Name          : sid
Designation   : manager
Basic Salary  : 100000.00
HRA (15%)     : 15000.00
DA (20%)      : 20000.00
Gross Salary  : 135000.00
PF (10%)      : 10000.00
LIC (8%)      : 8000.00
Total Ded.    : 18000.00
Net Salary    : 117000.00
-----

=====
ORGANIZATIONAL SALARY SUMMARY
=====
Total Employees Processed : 1
Total Gross Payout        : 135000.00
Total Net Payout          : 117000.00
=====

...Program finished with exit code 0
Press ENTER to exit console.
```